

Semana 2. Python para Ciencia de Datos

Curso: Fundamentos para Inteligencia Artificial — Especialización en Inteligencia Artificial (UNIMINUTO)

Duración: 90 minutos | **Modalidad:** virtual

Herramientas: Google Colab / Jupyter Notebook

Docente: Cesar Alfonso Bolado Silva

Resultados de aprendizaje de la sesión

- Relaciona el lenguaje de programación Python mediante librerías para inteligencia artificial (RA1 del curso).
 - **Interpretación operativa de esta sesión:** aplica la librería **pandas** para cargar, explorar y limpiar un dataset real con estructura tabular, documentando cada transformación realizada.
-

0. Puente con la Semana 1

En la Semana 1 trabajamos con tipos de datos simples (`int`, `float`, `str`, `bool`) y estructuras de control. Antes de entrar en pandas, conviene recordar brevemente las estructuras de datos nativas de Python que sirven de base para todo lo que viene:

Estructura	Mutable	Ejemplo	Uso típico
<code>list</code>	Sí	<code>[1, 2, 3]</code>	Colección ordenada y modificable
<code>tuple</code>	No	<code>(1, 2, 3)</code>	Colección ordenada e inmutable
<code>dict</code>	Sí	<code>{"nombre": "Ana"}</code>	Pares clave-valor
<code>set</code>	Sí	<code>{1, 2, 3}</code>	Colección sin duplicados ni orden

Pandas construye sus dos estructuras principales (`Series` y `DataFrame`) por encima de estas ideas y de los arrays de NumPy: una `Series` es, conceptualmente, una lista de valores del mismo tipo con una etiqueta (índice) para cada uno; un `DataFrame` es una colección de `Series` que comparten el mismo índice, organizadas en columnas — como una hoja de cálculo o una tabla de base de datos.

1. ¿Por qué pandas y no solo NumPy?

NumPy (visto conceptualmente en la Semana 1 y en profundidad en la Semana 4) trabaja muy bien con **matrices homogéneas**: todos los elementos del mismo tipo. Pero los datos reales rara vez son así — una tabla de estudiantes tiene columnas de texto (nombre), números enteros (edad), decimales (nota) y categorías (programa) al mismo tiempo.

Pandas es la librería estándar de Python para el tratamiento de datos tabulares. Fue creada por Wes McKinney y su primera versión se publicó en 2008; está construida sobre NumPy, por lo que hereda su rendimiento, y añade:

- Dos estructuras de datos especializadas: **Series** (unidimensional) y **DataFrame** (tabular).
- Funciones para leer y escribir datos desde archivos CSV, Excel, JSON o bases de datos.
- Herramientas para filtrar, ordenar, agrupar y combinar datos sin necesidad de escribir bucles manuales.
- Funciones específicas para tratar valores faltantes (*missing values*).

Convención de importación (usada en el 100% de los notebooks que verán sobre pandas):

```
import pandas as pd
import numpy as np
```

2. Estructuras de datos: **Series** y **DataFrame**

2.1 Series

Una **Series** es un arreglo unidimensional con una etiqueta (índice) para cada valor.

```
notas = pd.Series([4.5, 3.8, 2.9, 5.0], index=["Ana", "Luis", "Marta", "Iván"])
```

Si no se especifica **index**, pandas asigna automáticamente posiciones numéricas (0, 1, 2, ...).

2.2 DataFrame

Un **DataFrame** es una tabla: varias **Series** (columnas) que comparten el mismo índice (filas).

```
estudiantes = pd.DataFrame({
    "nombre": ["Ana", "Luis", "Marta", "Iván"],
    "programa": ["Ing. Software", "Ing. Industrial", "Ing. Software", "Ing.
Sistemas"],
    "nota_final": [4.5, 3.8, 2.9, 5.0]
})
```

Idea clave: en una matriz de NumPy todos los elementos deben ser del mismo tipo. En un DataFrame, cada **columna** puede tener su propio tipo de dato, pero dentro de una misma columna todos los valores deben ser del mismo tipo.

3. Cargar datos reales: **read_csv**

En la práctica casi nunca se escriben los datos a mano: se cargan desde un archivo. La función más usada es **pd.read_csv()**.

```
df = pd.read_csv("datos/estudiantes.csv")
```

Parámetros que conviene conocer desde ya:

Parámetro	Para qué sirve
<code>sep</code>	Indica el separador de columnas si no es una coma (p. ej. <code>sep=";"</code>)
<code>header</code>	Indica qué fila contiene los nombres de columna (<code>header=None</code> si el archivo no tiene cabecera)
<code>names</code>	Permite asignar nombres de columna manualmente

Pandas también puede leer Excel (`read_excel`), JSON (`read_json`) y conectarse directamente a bases de datos — no las usaremos hoy, pero conviene que sepan que existen para cuando trabajen con datos institucionales reales.

Para guardar un DataFrame de vuelta a disco:

```
df.to_csv("datos/estudiantes_limpio.csv", index=False)
```

`index=False` evita que pandas guarde la columna de índice numérico como si fuera un dato más — un detalle pequeño pero que ensucia el archivo de salida si se olvida.

4. Explorar un DataFrame antes de tocarlo

Antes de limpiar o transformar cualquier dato, hay que **entender qué se tiene**. Estas son las funciones de exploración que se usan en prácticamente cualquier notebook de ciencia de datos:

```
df.head()          # primeras 5 filas
df.tail(3)         # últimas 3 filas
df.shape           # (número de filas, número de columnas)
df.columns         # nombres de las columnas
df.info()          # tipo de dato de cada columna y conteo de valores no nulos
df.describe()      # estadísticas básicas de las columnas numéricas (media,
desviación, cuartiles, min, max)
```

`df.describe()` solo opera sobre columnas numéricas por defecto — si su dataset es mayormente texto, no verán mucho ahí todavía (eso lo resolveremos con `value_counts()` más abajo).

5. Selección e indexación

5.1 Seleccionar una columna

```
df["nota_final"]    # devuelve una Series
df[["nombre", "nota_final"]] # devuelve un DataFrame con varias columnas
```

5.2 loc e iloc

- **loc**: selecciona por **etiqueta** (nombre de fila/columna).
- **iloc**: selecciona por **posición** numérica (igual que en una lista de Python).

```
df.loc[0, "nombre"]          # valor en la fila con etiqueta 0, columna "nombre"
df.iloc[0, 0]                # valor en la primera fila, primera columna
df.loc[0:2, ["nombre", "nota_final"]] # varias filas y columnas por etiqueta
```

5.3 Filtrado booleano (la herramienta más usada en la práctica)

En vez de indicar posiciones, se puede indicar una **condición lógica**: pandas selecciona las filas donde la condición es verdadera.

```
df[df["nota_final"] >= 3.0]          # solo estudiantes que aprobaron
df[(df["nota_final"] >= 3.0) & (df["programa"] == "Ing. Software")] # condición
compuesta
```

Con múltiples condiciones se usa **&** (y) y **|** (o) — no las palabras **and/or** de Python puro, que no funcionan elemento a elemento sobre una Series.

6. Valores nulos (missing values)

Un dato faltante en pandas se representa como **NaN** (*Not a Number*). Antes de calcular cualquier estadística o entrenar cualquier modelo, hay que decidir qué hacer con ellos.

6.1 Detectarlos

```
df.isnull()          # True/False por cada celda
df.isnull().sum()    # cuántos nulos hay por columna – el primer chequeo que se
hace siempre
```

6.2 Eliminarlos

```
df.dropna()          # elimina cualquier fila que tenga al menos un
NaN
df.dropna(axis=1)     # elimina columnas con al menos un NaN
df.dropna(thresh=3)   # conserva filas con al menos 3 valores no nulos
```

6.3 Imputarlos (rellenarlos)

```
df.fillna(0) # rellena todos los NaN con un
valor fijo
df["nota_final"].fillna(df["nota_final"].mean()) # rellena con la media de la
columna
df.fillna(method="ffill") # propaga el último valor válido
hacia adelante
```

Advertencia pedagógica (importante): rellenar con la media solo tiene sentido en columnas numéricas. Si se intenta calcular la media de una columna de texto, pandas no sabrá qué hacer — la limpieza de datos casi siempre requiere tratar cada columna según su tipo, no aplicar una única regla a todo el DataFrame.

7. Conocer las categorías: `value_counts`

Para columnas de texto o categóricas, la función más usada para "entender qué hay ahí" es `value_counts()`:

```
df["programa"].unique() # lista de valores distintos, sin repetir
df["programa"].value_counts() # cuántas veces aparece cada valor distinto
```

Esto es, en la práctica, la forma más rápida de detectar errores de captura (por ejemplo, "Ing. Software" y "ing software" contados como categorías distintas).

8. Aplicar funciones propias: `apply`

Cuando ninguna función de pandas hace exactamente lo que se necesita, se puede aplicar una función propia a cada elemento de una columna:

```
def aprobo(nota):
    return "Aprobado" if nota >= 3.0 else "Reprobado"

df["estado"] = df["nota_final"].apply(aprobo)
```

`apply` ejecuta la función sobre cada valor de la Series (o cada fila/columna de un DataFrame) y devuelve el resultado como una nueva Series — que aquí guardamos como una columna nueva.

9. Cierre y anticipo (no se cubre en profundidad hoy)

Combinar tablas distintas con `merge()` (equivalente a un `JOIN` de SQL) y resumir datos por categoría con `groupby()`. Se mencionan aquí solo para que el vocabulario no sea nuevo cuando se retomen con más profundidad:

```
# Adelanto – se retoma con ejemplos completos en la Semana 3
pd.merge(tabla_1, tabla_2, on="columna_comun", how="left")
df.groupby("programa")["nota_final"].mean()
```

Entregable de la sesión

Dataset sugerido (proporcionado por el docente): dataset Titanic, alojado en el repositorio oficial de pandas — estable, real y con nulos genuinos (no fabricados) en tres columnas con patrones distintos, ideal para justificar decisiones de limpieza distintas por columna.

```
df = pd.read_csv("https://raw.githubusercontent.com/pandas-
dev/pandas/master/doc/data/titanic.csv")
```

891 filas × 12 columnas (**PassengerId**, **Survived**, **Pclass**, **Name**, **Sex**, **Age**, **SibSp**, **Parch**, **Ticket**, **Fare**, **Cabin**, **Embarked**). Nulos verificados: **Age** (177), **Cabin** (687), **Embarked** (2).

El estudiante que prefiera usar un dataset propio puede hacerlo, siempre que cumpla el mínimo de 5 columnas y 30 filas.

Notebook individual en Google Colab que:

1. Cargue el dataset (el sugerido u otro real elegido por el estudiante, con al menos 5 columnas y 30 filas).
2. Explore el dataset con **head()**, **shape**, **info()** y **describe()**.
3. Detecte y trate los valores nulos, justificando en una celda de texto (Markdown) la decisión tomada para cada columna (eliminar vs. imputar, y por qué).
4. Cree al menos una columna nueva usando **apply()**.

Criterios de evaluación:

- Ejecuta sin error.
- Las decisiones de limpieza están documentadas y son razonables para el tipo de dato de cada columna.
- El código es legible (nombres de variables claros).

Bibliografía y recursos audiovisuales

AfiEscuela. (2020, 10 de junio). *Curso Gratuito de Python para el tratamiento de datos - Clase 1/2* [video].

YouTube. <https://www.youtube.com/watch?v=MyeCc-Xwi5g>

AfiEscuela. (2020, 10 de junio). *Curso Gratuito de Python para el tratamiento de datos - Clase 2/2* [video].

YouTube. <https://www.youtube.com/watch?v=vnBjUEKl4Vw>

Industry S.F. (2021, 2 de julio). *Python para principiantes aplicado a Ciencia de Datos* [video]. YouTube.

<https://www.youtube.com/watch?v=5Vn2iKwDx6Q>

McKinney, W. (2023). *Python para análisis de datos: Manipulación de datos con pandas, NumPy y Jupyter* (3.^a ed.). Anaya Multimedia.

pandas development team. (s. f.). *pandas documentation*. <https://pandas.pydata.org/docs/>